

# Random Forest (Rastgele Orman) — Detaylı Konu Anlatımı

Sınav Çalışma Notları

## Contents

|                                                                               |   |
|-------------------------------------------------------------------------------|---|
| 1. Neden Tek Bir Karar Ağacı Yetmez? (Varyans Problemi)                       | 1 |
| 2. Bootstrap Örnekleme (Bagging'in İlk Ayağı)                                 | 2 |
| 3. Temel Yapı Taşı: Karar Ağacı Nasıl Kurulur?                                | 2 |
| 3.1 Saflık (Impurity) Ölçütü: Entropi                                         | 2 |
| 3.2 En İyi Bölünmeyi Bulmak: Bilgi Kazancı (Information Gain)                 | 3 |
| 3.3 Ağacın Büyütülmesi (Özyinelemeli Süreç)                                   | 3 |
| 4. Random Forest Algoritması — Tüm Adımlar Bir Arada                          | 3 |
| 5. $n_{\text{estimators}}$ (Ağaç Sayısı) Etkisi ve Kararlılık                 | 4 |
| 6. Out-of-Bag (OOB) Skoru — Ekstra Doğrulama Seti Gerektirmeyen Değerlendirme | 4 |
| 7. Özellik Önemi (Feature Importance)                                         | 4 |
| 7.1 Gini/Entropi Tabanlı (Varsayılan) Yöntem                                  | 4 |
| 7.2 Permütasyon Önemi (Permutation Importance) — Daha Güvenilir Alternatif    | 4 |
| 8. Hiperparametreler — Sınav İçin Özet Tablo                                  | 5 |
| 9. Random Forest'in Diğer Yöntemlerle Karşılaştırması                         | 5 |
| 10. Sınavda Sık Sorulan Kavramlar — Hızlı Hatırlatma                          | 6 |

## 1. Neden Tek Bir Karar Ağacı Yetmez? (Varyans Problemi)

Karar ağaçları **güçlü ama kararsız (unstable)** modellerdir. “Kararsız” derken kastedilen şudur: eğitim verisinde yapılacak küçük bir değişiklik (birkaç satırın çıkarılması, eklenmesi ya da değiştirilmesi), tamamen farklı bir ağaç yapısına yol açabilir. Bu duruma **yüksek varyans** denir.

Bias-Variance (yanlılık-varyans) çerçevesinde düşünersek:

- **Bias (yanlılık):** Modelin gerçek ilişkiyi ne kadar iyi yakalayabildiği. Derin bir karar ağacı, eğitim verisine çok iyi uyum sağlayabildiği için **düşük yanlılığa** sahiptir.

- **Varyans:** Modelin farklı eğitim örneklemeleri arasında ne kadar değiştiği. Derin bir tek karar ağacı, veri setindeki küçük gürültüye bile aşırı duyarlı olduğu için **yüksek varyansa** sahiptir.

Sonuç: Tek bir derin karar ağacı, eğitim verisinde mükemmel yakın performans gösterse de (düşük bias), test/yeni veride tutarsız ve genelleme kabiliyeti zayıf bir model olur (yüksek varyans → overfitting).

**Random Forest’in temel motivasyonu:** Birbirine benzer ama tam olarak aynı olmayan çok sayıda ağaç kurup bunların ortalamasını/oylamasını almak, bireysel ağaçların varyansını “söndürür” — çünkü rastgele hatalar farklı yönlerde olma eğilimindedir ve ortalama alındığında birbirini götürür. Yanlılık fazla artmadan varyans azaltılmış olur.

## 2. Bootstrap Örnekleme (Bagging’in İlk Ayağı)

**Bootstrap örnekleme**, orijinal veri setinden **yerine koyarak (with replacement)** rastgele örnekler çekmek işlemidir. N satırlık bir veri setinden yine N satırlık bir bootstrap örneklem çekildiğinde:

- Bazı satırlar birden fazla kez seçilebilir.
- Bazı satırlar hiç seçilmeyebilir.

**Matematiksel Gerçek — Out-of-Bag (OOB) Oranı:** Bir satırın tek bir çekilişte seçilmeme olasılığı  $(1 - 1/N)^N$ ’dir. N çekiliş sonunda hiç seçilmeme olasılığı:

$$P(\text{seçilmedi}) = \left(1 - \frac{1}{N}\right)^N \xrightarrow{N \rightarrow \infty} e^{-1} \approx 0.368$$

Yani her bootstrap örnekleminde, orijinal verinin **yaklaşık %36,8’i dışarıda kalır**. Bu dışarıda kalan veriler “Out-of-Bag (OOB)” olarak adlandırılır ve önemli bir doğrulama fırsatı sunar (bkz. Bölüm 6).

**Bagging (Bootstrap Aggregating):** Bu isim, “bootstrap” + “aggregating” (birleştirme) kelimelerinin kısaltmasıdır: çok sayıda bootstrap örnekleme üzerinde ayrı ayrı model eğitip tahminlerini birleştirme stratejisidir. Random Forest, bagging’in üzerine bir katman daha ekler: **rastgele özellik seçimi**.

## 3. Temel Yapı Taşı: Karar Ağacı Nasıl Kurulur?

### 3.1 Safılık (Impurity) Ölçütü: Entropi

Bir düğümdeki sınıf karışıklığını ölçmek için **entropi** kullanılır:

$$H(y) = - \sum_{i=1}^k p_i \log_2(p_i)$$

Burada  $p_i$ , düğümdeki  $i$ . sınıfın oranıdır. Entropi:

- Tüm örnekler tek bir sınıftaysa →  $H = 0$  (tamamen saf).

- Sınıflar eşit dağılmışsa → H maksimum değerine ulaşır (en karışık durum).

Alternatif olarak **Gini impurity** de kullanılabilir:  $Gini = 1 - \sum p_i^2$ . İkisi de benzer davranış sergiler; Gini hesaplama olarak biraz daha hızlıdır.

### 3.2 En İyi Bölünmeyi Bulmak: Bilgi Kazancı (Information Gain)

Bir düğümü ikiye bölmek için, tüm özellik-eşik kombinasyonları denenir ve **bilgi kazancını** maksimize eden bölünme seçilir:

$$IG = H(\text{ebeveyn}) - \left( \frac{n_{\text{sol}}}{n} H(\text{sol}) + \frac{n_{\text{sag}}}{n} H(\text{sag}) \right)$$

**Random Forest'a Özgü Fark — max\_features:** Normal bir karar ağacında her bölünmede **tüm özellikler** değerlendirilir. Random Forest'ta ise her bölünmede özelliklerin **rastgele bir alt kümesi** (max\_features, sınıflandırmada genelde  $\sqrt{p}$ , regresyonda genelde  $p/3$ ) değerlendirilir. Bu, ağaçlar arasındaki **korelasyonu azaltmak** için kritik bir adımdır — eğer tüm ağaçlar her seferinde en güçlü özelliği kullanırsa, ağaçlar birbirine çok benzer olur ve bagging'in varyans azaltma etkisi zayıflar.

### 3.3 Ağacın Büyütülmesi (Özyinelemeli Süreç)

Bir ağaç, kök düğümünden başlayarak şu adımlarla özyinelemeli (recursive) olarak kurulur:

1. Mevcut düğümdeki verilerle en iyi bölünmeyi bul (rastgele özellik alt kümesi üzerinden).
2. Veriyi bu bölünmeye göre sol ve sağ alt düğümlere ayır.
3. Her alt düğüm için 1-2 adımlarını tekrarla.
4. **Durdurma kriterlerine** ulaşıncaya düğüm bir yaprak (leaf) olur:
  - max\_depth'e ulaşıldı,
  - düğümdeki örnek sayısı min\_samples\_split'ten az,
  - düğüm zaten saf (entropy = 0).

## 4. Random Forest Algoritması — Tüm Adımlar Bir Arada

1. n\_estimators kadar ağaç kurulacak.
2. **Her ağaç için:**
  - a. Orijinal veriden bir **bootstrap örnekleme** çek.
  - b. Bu örneklem üzerinde, her bölünmede rastgele bir özellik alt kümesi (max\_features) kullanarak bir karar ağacı kur (genelde max\_depth ile sınırlandırılmış).
3. **Tahmin aşaması:**
  - Sınıflandırma: her ağacın tahmini alınır, **çoğunluk oylaması (majority voting)** yapılır.
  - Regresyon: her ağacın tahmininin **ortalaması** alınır.

**İki Kaynaklı Rastgelelik:** (1) Bootstrap örnekleme (satır bazında rastgelelik), (2) max\_features (sütun bazında rastgelelik). Bu iki rastgelelik kaynağı birlikte, ağaçlar arasındaki korelasyonu düşürerek topluluğun (ensemble) varyansını önemli ölçüde azaltır.

## 5. n\_estimators (Ağaç Sayısı) Etkisi ve Kararlılık

n\_estimators=1 iken model aslında **tek bir bootstrap ağacına** eşdeğerdir ve tekrar eden denemelerde yüksek varyans gösterir (her defasında farklı bir sonuç). Ağaç sayısı arttıkça:

- Topluluğun ortalama tahmini **kararlılaşır** (varyans azalır).
- Belirli bir noktadan sonra (n\_estimators genelde 100-500 arası) ek ağaçlar **marjinal fayda sağlamaz** — yani model performansı platoya ulaşır ama hesaplama maliyeti artmaya devam eder.
- **Önemli:** n\_estimators artışı, Gradient Boosting'in aksine **overfitting riskini artırmaz** (çünkü ağaçlar bağımsız ve paralel kurulur, birbirinin hatasını "ezberlemez"). Bu, Random Forest'in en pratik özelliklerinden biridir — n\_estimators'ı makul bir üst sınıra kadar artırmak neredeyse her zaman güvenlidir.

---

## 6. Out-of-Bag (OOB) Skoru — Ekstra Doğrulama Seti Gerek-tirmeyen Değerlendirme

Bölüm 2'de gördüğümüz gibi, her bootstrap örnekleme orijinal verinin ~%36,8'ini dışarıda bırakır. Bu, güçlü bir fırsat sunar:

### OOB Skoru Nasıl Hesaplanır?

1. Her satır  $x_i$  için,  $x_i$ 'nin bootstrap örnekleminde **yer almadığı** tüm ağaçlar belirlenir.
2. Bu ağaçların  $x_i$  üzerindeki tahminleri toplanır (oylama/ortalama).
3. Bu tahmin,  $x_i$ 'nin gerçek etiketiyle karşılaştırılır.
4. Tüm satırlar için bu işlem tekrarlanır ve genel bir doğruluk/hata skoru hesaplanır.

**Neden Önemli?** OOB skoru, ayrı bir doğrulama (validation) seti ayırmadan, **eğitim verisinin kendisini kullanarak** dürüst bir genelleme performansı tahmini sağlar — özellikle veri az olduğunda değerlidir. scikit-learn'de oob\_score=True parametresiyle otomatik hesaplanır.

---

## 7. Özellik Önemi (Feature Importance)

### 7.1 Gini/Entropi Tabanlı (Varsayılan) Yöntem

Her özelliğin, tüm ağaçlardaki tüm bölünmelerde sağladığı **ortalama saflık azalması (mean decrease in impurity)** toplanarak hesaplanır. Hızlı hesaplanır ama iki önemli **yanlılık (bias)** kaynağı vardır:

- **Yüksek kardinaliteli (çok farklı değer alan) sayısal değişkenler** yapay olarak daha önemli görünebilir (çünkü daha fazla potansiyel bölünme noktası sunarlar).
- Sadece **eğitim verisi** üzerinden hesaplanır — aşırı öğrenmiş bir model, gürültülü bir özelliğe de yüksek önem verebilir.

### 7.2 Permütasyon Önemi (Permutation Importance) — Daha Güvenilir Alternatif

**Mantık:** Bir özelliğin gerçekten önemli olup olmadığını anlamak için, o özelliğin değerlerini **rastgele karıştırıp (permüte edip)** model performansının ne kadar düştüğüne bakılır.

**Adımlar:**

1. Test/doğrulama setinde modelin **başlangıç (baseline) doğruluğu** hesaplanır.
2. Bir özelliğin sütunundaki değerler rastgele karıştırılır (böylece o özellik ile hedef arasındaki ilişki bozulur, ama değişkenin marjinal dağılımı korunur).
3. Model bu karıştırılmış veri üzerinde yeniden değerlendirilir.
4. **Önem = Baseline doğruluk – Karıştırılmış veri doğruluğu**. Büyük bir düşüş, özelliğin önemli olduğunu gösterir.
5. Gürültüyü azaltmak için işlem birkaç kez (n\_repeats) tekrarlanıp ortalaması alınır.

**Avantajı:** Model-agnostiktir (herhangi bir modelle kullanılabilir), test verisi üzerinde hesaplanabilir (gerçek genelleme önemini yansıtır), yüksek kardinaliteli değişken yanlılığına sahip değildir.

## 8. Hiperparametreler — Sınav İçin Özet Tablo

| Parametre         | Anlamı                                               | Artınca Ne Olur?                                                         |
|-------------------|------------------------------------------------------|--------------------------------------------------------------------------|
| n_estimators      | Ağaç sayısı                                          | Varyans azalır, hesaplama maliyeti artar (overfitting riski artmaz)      |
| max_depth         | Her ağacın maksimum derinliği                        | Bias azalır, varyans artar (aşırı derin → overfitting)                   |
| max_features      | Her bölünmede değerlendirilen özellik sayısı         | Azsa ağaçlar arası korelasyon düşer (iyi); çoksa RF, bagging'e yaklaşır  |
| min_samples_split | Bir düğümün bölünmesi için gereken min. örnek sayısı | Artınca ağaçlar sığlaşır, overfitting azalır                             |
| min_samples_leaf  | Bir yaprakta olması gereken min. örnek sayısı        | Artınca model daha genelleyici (daha az overfitting) olur                |
| class_weight      | Dengesiz sınıflandırmada sınıf ağırlıklandırma       | 'balanced' → azınlık sınıfın etkisi artırılır                            |
| bootstrap         | Bootstrap örnekleme kullanılsın mı                   | False yapılırsa tüm veri her ağaçta kullanılır (bagging etkisi kaybolur) |

## 9. Random Forest'ın Diğer Yöntemlerle Karşılaştırması

| Kriter            | Tek Karar Ağacı               | Random Forest                                |
|-------------------|-------------------------------|----------------------------------------------|
| Bias              | Düşük (derin ağaçta)          | Benzer (tek ağaca yakın)                     |
| Varyans           | Yüksek                        | Düşük (bagging + rastgele özellik sayesinde) |
| Eğitim stratejisi | Tek model                     | Paralel, bağımsız çok sayıda ağaç            |
| Yorumlanabilirlik | Yüksek (görselleştirilebilir) | Orta (feature importance ile)                |
| Overfitting riski | Yüksek                        | Düşük-orta                                   |

|                  |                 |                                        |
|------------------|-----------------|----------------------------------------|
| Kriter           | Tek Karar Ağacı | Random Forest                          |
| Parallelleştirme | Gerek yok       | Doğal olarak<br>parallelleştirilebilir |

**Random Forest vs Boosting Aileleri (AdaBoost/Gradient Boosting/XGBoost/LightGBM):**  
Random Forest **paralel + bağımsız** ağaçlarla **varyansı** azaltırken, boosting yöntemleri **sıralı + birbirine bağımlı** modellerle **yanlılığı (bias)** azaltır. Bu, iki ailenin en temel felsefi farkıdır.

## 10. Sınavda Sık Sorulan Kavramlar — Hızlı Hatırlatma

- **Bagging = Bootstrap + Aggregating.**
- Bootstrap örneklemede bir satırın seçilmeme olasılığı yaklaşık  $e^{-1} \approx \%36,8$ 'dir → OOB verisi buradan gelir.
- Random Forest'ta rastgelelik **iki kaynaktan** gelir: satır (bootstrap) ve sütun (max\_features).
- n\_estimators artışı overfitting riskini **artırmaz**, sadece hesaplama maliyetini artırır (Gradient Boosting/AdaBoost'un aksine).
- Gini/entropi tabanlı feature importance **yüksek kardinaliteli değişkenlere yanlı** olabilir; permütasyon önemi daha güvenilirdir.
- OOB skoru, ayrı bir doğrulama seti gerektirmeden genelleme performansı tahmini sağlar.
- Random Forest, dengesiz (imbalanced) veri setlerinde `class_weight='balanced'` parametresiyle uyarlanabilir.